

Church AI Foundation (CAIF) Engineering Guide

Version 3.0 Draft

1. Vision

1.1 Purpose

The **Church AI Foundation (CAIF)** is an open-source engineering initiative that explores how Artificial Intelligence can responsibly preserve, organize, retrieve, and apply church knowledge for ministry.

Rather than developing a single AI application, CAIF provides a shared engineering foundation upon which multiple church AI applications can be built collaboratively.

Examples include:

- CKAIS (Central Kitchen AI System)
- Biblical AI
- Sermon Intelligence
- Ministry Knowledge Management
- Mentoring Systems
- Future AI-assisted ministry applications

The Foundation itself is independent of any single church, allowing churches and Christian organizations to adopt, extend, or contribute according to their own needs.

1.2 Why CAIF?

Churches accumulate valuable knowledge over decades through:

- sermons,

- Bible studies,
- ministry experience,
- fellowship activities,
- administrative decisions,
- mission work,
- historical documents,
- photographs,
- videos,
- and the faithful service of volunteers.

Unfortunately, much of this knowledge becomes fragmented or disappears as leaders retire and volunteers change.

Artificial Intelligence provides a unique opportunity to preserve this institutional memory and make it more accessible to future generations.

CAIF exists to build the engineering foundation that makes this possible.

1.3 Mission

Mission Statement

To build trustworthy AI capabilities that preserve church knowledge and empower ministry through open collaboration.

The mission emphasizes four priorities:

- Preserve knowledge rather than merely storing data.
- Enable collaboration rather than isolated development.
- Build reusable capabilities rather than isolated applications.
- Ensure AI remains accountable to human leadership.

1.4 Scope

CAIF focuses on engineering infrastructure rather than church administration.

The Foundation does **not** define theology, replace pastoral leadership, or automate spiritual decision making.

Instead, it provides reusable capabilities that assist ministry while leaving doctrinal and pastoral authority in the hands of church leaders.

2. Design Principles

Technology evolves rapidly. The engineering principles of CAIF should remain stable across multiple generations of AI technologies.

Principle 1 — Mission Before Technology

Technology exists to serve ministry.

Engineering decisions should always be evaluated by ministry value rather than technical novelty.

Frameworks, programming languages, and AI models will change.

The mission of serving churches remains unchanged.

Principle 2 — Specification Before Implementation

Specifications are the primary engineering assets.

Rather than focusing primarily on source code, contributors should first define:

- system behavior,

- capability requirements,
- governance policies,
- evaluation criteria,
- engineering constraints.

Implementations may evolve many times.

Specifications should remain comparatively stable.

Principle 3 — Capabilities Before Applications

Applications solve immediate ministry problems.

Capabilities provide reusable engineering services.

Examples of capabilities include:

- Document Retrieval
- Citation Generation
- Bible Reference Parsing
- Governance
- Media Search
- Audit Logging

Applications should compose these capabilities rather than duplicate them.

This promotes reuse, consistency, and collaboration.

Principle 4 — Evidence Before Generation

AI should retrieve trustworthy evidence before generating conclusions whenever practical.

Evidence may include:

- church documents,
- meeting minutes,
- sermons,
- Bible passages,
- historical records,
- approved ministry policies.

Generated responses should distinguish clearly between:

- retrieved facts,
- AI-generated summaries,
- interpretation,
- and human decisions.

Principle 5 — Governance Before Deployment

Passing automated tests does not necessarily make an AI system trustworthy.

Governance includes:

- specification review,
- security,
- privacy,
- traceability,
- human approval,
- release management,
- theological oversight where appropriate.

Governance remains a human responsibility.

Principle 6 — Replace Technologies, Preserve Knowledge

CAIF intentionally separates long-term engineering assets from short-term implementation technologies.

Long-term assets include:

- specifications,
- knowledge,
- governance,
- capabilities,
- institutional memory.

Replaceable technologies include:

- AI models,
- agent runtimes,
- development tools,
- programming languages,
- deployment platforms.

This separation allows CAIF to evolve without repeatedly redesigning the architecture.

Principle 7 — Human Accountability

Artificial Intelligence assists human decision making.

It does not replace human responsibility.

Pastors remain responsible for pastoral care.

Elders remain responsible for church governance.

Developers remain responsible for software quality.

AI supports these roles rather than replacing them.

3. Architecture

3.1 Architectural Philosophy

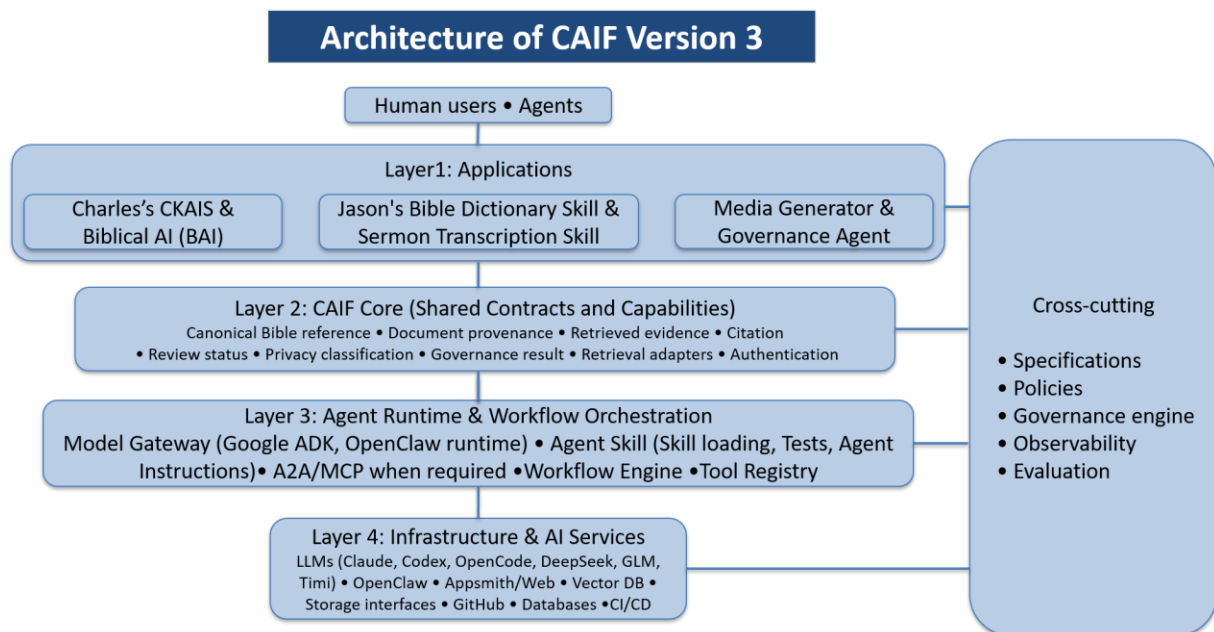
CAIF adopts a **Capability-Centric Architecture**.

Applications are intentionally separated from reusable engineering capabilities.

This separation allows independent teams to develop applications while sharing common engineering assets.

The architecture also separates long-term organizational knowledge from rapidly evolving AI technologies.

3.2 Overall Architecture



3.3 Ministry Applications

The upper layer contains ministry-oriented applications that directly serve church needs.

Examples include:

- CKAIS
- Biblical AI
- Sermon Intelligence
- Mentoring
- Church Administration
- Future ministry applications

Applications focus on user experience and ministry workflows.

They should avoid implementing common engineering functionality already available in the Shared Core.

3.4 Shared Capabilities

The Shared Capability Layer represents the permanent engineering foundation of CAIF.

Unlike applications, capabilities are intended for reuse across multiple projects.

Typical capabilities include:

- document retrieval,
- semantic search,
- citations,
- governance,
- evaluation,
- authentication,
- metadata management,

- media indexing,
- audit logging.

This layer is expected to evolve slowly and preserve compatibility whenever practical.

3.5 Agent Runtime

The Agent Runtime determines **how** capabilities are executed.

Examples of runtime technologies today include:

- Agent Skills,
- Google ADK,
- A2A,
- workflow engines,
- model gateways.

These technologies are implementation choices rather than architectural principles.

CAIF intentionally treats this layer as replaceable.

Future runtimes may be adopted without changing the upper architectural layers.

3.6 Infrastructure Services

The infrastructure layer contains the underlying technologies required to operate the system.

Examples include:

- Large Language Models,
- Vector Databases,
- Relational Databases,
- Object Storage,

- GitHub,
- Continuous Integration,
- Authentication Services.

These technologies support the upper layers but should remain hidden from application developers whenever possible.

3.7 Cross-Cutting Engineering Disciplines

Several engineering disciplines apply to every architectural layer.

These include:

- Specifications
- Governance
- Evaluation
- Security
- Privacy
- Observability
- Documentation

These disciplines are **not individual software modules**.

They define how every capability and application should be designed, implemented, reviewed, and maintained.

3.8 Architecture Evolution

One of the primary design goals of CAIF is architectural longevity.

Future AI technologies will continue to evolve rapidly.

Examples include:

- coding agents,
- runtime frameworks,
- orchestration systems,
- language models,
- deployment platforms.

CAIF therefore preserves stable architectural concepts while allowing implementation technologies to evolve independently.

The objective is to ensure that church knowledge, engineering specifications, and governance policies remain valuable long after today's technologies have been replaced.

4. Development Model

4.1 Agentic Development

Software engineering is entering a new era in which AI assists not only application users but also software developers. Coding agents can now generate implementations, tests, documentation, and pull requests from well-defined specifications.

CAIF adopts this **Agentic Development Model**, where human contributors focus on defining *what* should be built, while AI agents increasingly assist in determining *how* it is implemented.

The role of software engineers therefore shifts from primarily writing code to designing systems, organizing knowledge, defining specifications, and governing AI-generated artifacts.

4.2 Engineering Workflow

CAIF follows the engineering workflow shown below.

Vision → Specification → Capability Design → AI-assisted Implementation
→ Testing & Evaluation → Governance Review → Release

Unlike traditional development, implementation is no longer the primary engineering activity. Instead, specifications become the foundation from which implementations are generated, tested, and continuously improved.

4.3 Engineering Artifacts

Every CAIF capability should maintain a common set of engineering artifacts.

Artifact	Purpose
README.md	Introduction and usage
SPECIFICATION.md	Functional requirements
SKILL.md	Runtime agent description
AGENTS.md	Coding-agent instructions
Source Code	Implementation
Tests	Functional verification
Evaluation	Quality measurement
CHANGELOG	Version history

These artifacts are treated as engineering assets rather than supplementary documentation.

4.4 Human and AI Responsibilities

CAIF intentionally separates responsibilities between human contributors and AI agents.

Human Contributors	AI Agents
Vision	Code generation
Architecture	Unit tests
Specifications	Documentation updates
Knowledge organization	Refactoring
Governance	Regression checking
Final approval	Routine implementation

Human judgment remains essential for architecture, governance, theology, and ministry.

5. CAIF Shared Core

5.1 Purpose

The CAIF Shared Core provides reusable engineering capabilities shared by multiple applications.

Rather than duplicating retrieval, governance, citation, or evaluation logic in every application, these capabilities are developed once and reused across the Foundation.

Applications therefore remain relatively small while the Shared Core gradually matures into a long-term engineering asset.

5.2 Shared Core Components

The Shared Core consists of six major capability groups.

Component	Responsibility
Specifications	Shared engineering definitions
Contracts	Common semantics between capabilities
Retrieval	Search, indexing, citations
Governance	Policies, approval, audit
Evaluation	Quality assessment
Observability	Logging and operational insight

These components evolve more slowly than individual applications and provide the stable foundation upon which applications are built.

5.3 Shared Specifications

Specifications define expected behavior independently of implementation.

Examples include:

- retrieval behavior,
- metadata conventions,
- privacy requirements,
- evaluation criteria,
- ministry requirements.

Specifications are the primary engineering reference for both human developers and coding agents.

5.4 Shared Contracts

Contracts define the common language used between capabilities.

Typical contracts include:

- Retrieved Evidence
- Bible Reference
- Citation
- Review Decision
- Audit Record

Contracts define the **meaning** of exchanged information rather than communication protocols.

This semantic consistency allows independently developed capabilities to cooperate reliably.

5.5 Retrieval

Retrieval is one of the most frequently reused capabilities.

Responsibilities include:

- document search,
- semantic retrieval,
- metadata filtering,
- citation generation,
- permission checking.

Applications may extend retrieval for domain-specific needs while sharing the common retrieval infrastructure.

5.6 Governance

Governance ensures that AI systems remain trustworthy throughout development and

operation.

Typical governance functions include:

- specification review,
- approval workflow,
- audit logging,
- release policy,
- privacy protection,
- theological review where appropriate.

Governance applies across every application rather than belonging to any single project.

5.7 Evaluation and Observability

Evaluation measures capability quality.

Typical metrics include:

- retrieval accuracy,
- citation correctness,
- response quality,
- hallucination rate,
- governance compliance.

Observability records operational behavior, including:

- capability usage,
- execution latency,
- model utilization,
- workflow history,

- approval records.

Together they provide continuous feedback for improving the Foundation.

6. Capability Model

6.1 Capability-Centric Engineering

CAIF is organized around reusable **Capabilities** rather than complete applications.

Applications solve ministry-specific problems.

Capabilities solve reusable engineering problems.

This distinction encourages collaboration while minimizing duplicated effort.

6.2 Definition of a Capability

A capability is an independently maintained engineering component that:

- performs one primary function,
- can be reused by multiple applications,
- exposes stable semantics,
- evolves independently.

Examples include:

- Church History Retrieval
- Bible Reference Parsing
- Citation Service
- Media Memory
- Governance

- Document Ingestion
-

6.3 Capability Lifecycle

Every capability follows the same lifecycle.

Idea → Specification → Implementation → Testing → Evaluation → Governance → Release

The lifecycle emphasizes continuous improvement rather than one-time delivery.

6.4 Capability Structure

Each capability should maintain a consistent internal organization.

```
Capability/  
  
README.md  
  
SPECIFICATION.md  
  
SKILL.md  
  
AGENTS.md  
  
implementation/  
  
tests/  
  
evaluation/
```

This standard organization enables both human contributors and coding agents to understand unfamiliar capabilities quickly.

6.5 Capability Ownership

Every capability has a maintainer responsible for:

- reviewing contributions,
- maintaining specifications,

- approving releases,
- coordinating future enhancements.

Ownership provides accountability without preventing collaborative development.

6.6 Capability Composition

Applications are constructed by composing multiple capabilities.

Example:

CKAIS

- └─ Document Retrieval
- └─ Church History
- └─ Media Memory
- └─ Citation
- └─ Governance

Biblical AI may compose:

Biblical AI

- └─ Bible Retrieval
- └─ Canonical Interpretation
- └─ Citation
- └─ Governance
- └─ Evaluation

Because capabilities are shared, improvements automatically benefit multiple applications.

6.7 Capability Evolution

Capabilities should evolve independently of applications whenever practical.

Implementations may adopt new technologies while preserving:

- specifications,
- contracts,
- semantics,

- governance,
- compatibility.

Applications therefore remain stable even as underlying implementations improve.

6.8 Capability Registry

All reusable capabilities should be recorded in a shared registry.

Each entry identifies:

- capability name,
- version,
- maintainer,
- status,
- dependencies,
- documentation.

The registry enables contributors and future agent platforms to discover available capabilities without needing to inspect application source code.

7. Repository Organization

7.1 Purpose

The GitHub repository is the primary engineering workspace of the Church AI Foundation (CAIF). It stores not only source code, but also specifications, documentation, governance policies, evaluation assets, and reusable capabilities. Repository organization should enable both human contributors and AI coding agents to understand, maintain, and extend the project efficiently.

7.2 GitHub Organization

CAIF is developed under the **QIA (Quantum Intelligence Association)** GitHub organization. Individual applications remain independent repositories while sharing a common engineering foundation.

```
qiassoc/  
├── CAIF  
├── CKAIS  
├── BiblicalAI  
├── SermonAI  
├── Mentor  
└── Other Applications
```

This organization allows each project to evolve independently while encouraging reuse through CAIF Shared Core.

7.3 Repository Structure

The CAIF repository contains the shared engineering assets of the Foundation.

```
CAIF/  
  
README.md  
VISION.md  
ARCHITECTURE.md  
ROADMAP.md  
CONTRIBUTING.md  
AGENTS.md  
LICENSE  
  
shared/  
capabilities/  
governance/  
examples/  
tests/  
docs/
```

Repository documentation is considered an engineering artifact and should evolve together with the implementation.

7.4 Engineering Standards

All reusable capabilities should follow common engineering practices:

- maintain clear specifications;
- provide documentation and examples;
- include automated tests;
- follow semantic versioning;
- support governance review before release.

Repository organization should favor clarity, modularity, and long-term maintainability over framework-specific conventions.

8. Governance

8.1 Purpose

Governance ensures that CAIF remains trustworthy as both the project and AI technologies evolve.

Passing automated tests alone is insufficient. Every reusable capability should also satisfy engineering, security, and organizational requirements before becoming part of the Foundation.

Governance therefore applies throughout the entire development lifecycle.

8.2 Governance Areas

CAIF governance includes five complementary areas.

Area	Responsibility
Specification	Completeness and consistency of requirements
Engineering	Code quality, testing, documentation
Security & Privacy	Protection of confidential information

Area	Responsibility
Release	Version control, approval, traceability
Ministry	Human review for church-specific or doctrinal content

These governance activities protect both software quality and ministry integrity.

8.3 Human Responsibility

AI agents may generate code, documentation, and tests, but responsibility remains with human contributors.

Within CAIF:

- contributors maintain specifications;
- maintainers review changes;
- release managers approve production releases;
- pastors and ministry leaders retain responsibility for theological and ministry decisions.

AI assists engineering; it does not replace accountability.

9. Roadmap

9.1 Development Strategy

CAIF follows an incremental development strategy. Rather than attempting to build a comprehensive platform from the beginning, the Foundation evolves through a sequence of practical prototypes.

Each stage should produce usable software while establishing reusable engineering assets for future development.

9.2 Stage 1 – Foundation

Objectives:

- establish GitHub organization;
- define engineering standards;
- create Shared Core structure;
- adopt Agentic Development workflow;
- develop initial reusable capabilities.

Representative technologies may include Agent Skills, Google ADK, DeepSeek, or other contemporary agent frameworks. These technologies are implementation choices rather than permanent architectural commitments.

9.3 Stage 2 – Initial Applications

Develop the first ministry-oriented applications using the Shared Core.

Initial candidates include:

- CKAIS
- Biblical AI
- Sermon Intelligence

The primary objective is to validate that independently developed applications can successfully reuse common capabilities while maintaining consistent governance.

9.4 Stage 3 – Community Growth

Expand participation beyond the initial development team.

Goals include:

- onboarding new contributors;
- extending capability libraries;
- improving documentation;
- strengthening evaluation and governance;
- supporting additional church ministries.

At this stage, the Foundation becomes a collaborative open-source community rather than a small engineering project.

9.5 Long-Term Vision

CAIF is designed to evolve independently of specific AI technologies.

Future development may adopt new:

- agent runtimes,
- orchestration frameworks,
- language models,
- retrieval technologies,
- deployment platforms.

Regardless of technological evolution, the Foundation will continue preserving its core engineering assets:

- specifications,
- capabilities,
- governance,
- institutional knowledge,
- and reusable ministry experience.

The long-term success of CAIF will be measured not by the technologies it adopts, but

by the knowledge it preserves and the churches it enables to serve more effectively.

10. Initial Team Roles

Role	Initial Owner	Responsibility
Architecture	Charles	Vision, architecture, roadmap
Agent Development	Jason	Agent Skills, runtime, coding workflow
Governance	Zhihan	Release governance, compliance, traceability
Application	Shared	CKAIS, Biblical AI, Sermon AI
Community	Shared	Documentation, onboarding, reviews

11. Documentation Architecture

The following is the CAIF documentation hierarchy:

Document	Audience	Length
Vision Brief	Pastors, elders, church leadership	3 pages
Project Guide	Contributors (Charles, Jason, Zhihan, future developers)	26 pages
Engineering Reference	Architects and maintainers	No page limit
GitHub Markdown	Daily development	Living documents